What is Spring Boot?

Spring Boot is an open-source Java framework supported by Apache developers, which provides a pre-configured environment for developers for easy startup of new applications, reducing the time investment for configuration and setup. It was the project of the Pivotal team to make the configurations and setup processes easy, hence, even more time could be invested into writing code rather than configuration. Spring Boot's main goal is to reduce both unit test time and integration test time for most developers. This is what makes it very unique; it can do a lot of advanced things without the need for much XML configuration.

Why Choose Spring Boot?

Spring Boot facilitates application development in the real world by offering a suite of tools, supported by a wide range of frameworks that developers can practically work with. Features such as embedded servers, auto-configuration, and firm security defaults bring Spring Boot up as one of the best options for quickly getting applications out in production and making it a strong aid for developers, too.

Key Benefits of Spring Boot:

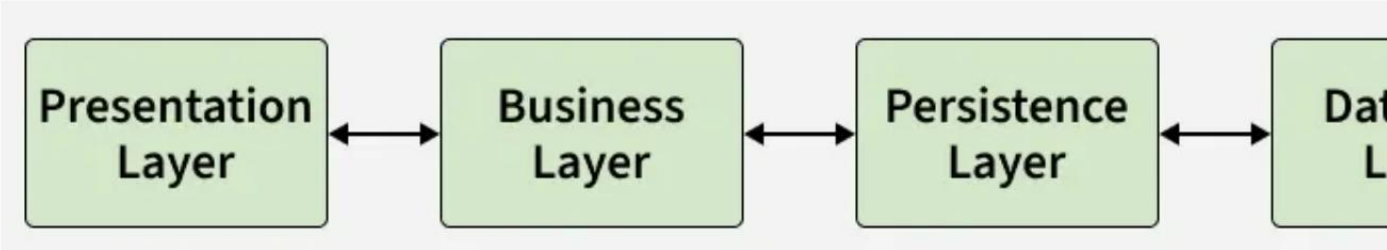
- **Easy to use: Auto-configuration available in Spring Boot, coupled with embedded servers, hugely lessens the complexity associated with setting up new projects.**
- **Microservices-Friendly: Built-in, it essentially sets an ideal environment for any microservice development with its lightweight and modular architecture.**
- **Community and Support: The large community and extensive documentation of the Go language community make it easier for you to find possible solutions along with best practices.**
- **Production Ready: It has built-in tools for monitoring, security, and configuration, with Spring Boot applications ready for deployment.**

Spring Boot - Architecture

Spring Boot is a framework built on top of the Spring Framework that simplifies application development by providing auto-configuration, embedded servers, and production-ready features. Its architecture is designed to reduce boilerplate code, making it easier to build standalone, scalable, and maintainable Java applications

Spring Boot Architecture Layers

Spring Boot consists of the following four layers:



Spring Boot Architecture Layers

1. Presentation Layer

The Presentation Layer acts as the entry point of the Spring Boot application, also known as the controller layer. It is responsible for handling all incoming HTTP requests and sending appropriate responses back to the client.

Responsibilities:

- Handles HTTP methods such as GET, POST, PUT, DELETE
- Exposes RESTful APIs using controllers
- Performs request validation
- Manages authentication entry points
- Converts Java objects to JSON and vice versa
- Forwards validated requests to the Business Layer

Common Components:

- @RestController / @Controller
- @RequestMapping, @GetMapping, @PostMapping
- @RequestBody, @PathVariable

2. Business Layer

The Business Layer, also known as the Service Layer, contains the core business logic of the application. It acts as an intermediary between the Presentation and Persistence layers.

Responsibilities:

- Implements business rules and workflows
- Processes and validates data
- Handles authentication and authorization logic (using Spring Security if required)
- Manages transactions using @Transactional

- Communicates with the Persistence Layer to retrieve or store data

Common Components:

- @Service
- @Transactional

3. Persistence Layer

The Persistence Layer is responsible for database interaction and data access logic. It abstracts the underlying database operations from the rest of the application.

Responsibilities:

- Maps Java objects to database tables using ORM frameworks
- Performs CRUD (Create, Read, Update, Delete) operations
- Manages database transactions
- Supports both relational and NoSQL databases

Technologies Used:

- Spring Data JPA
- Hibernate
- R2DBC (for reactive applications)

Common Components:

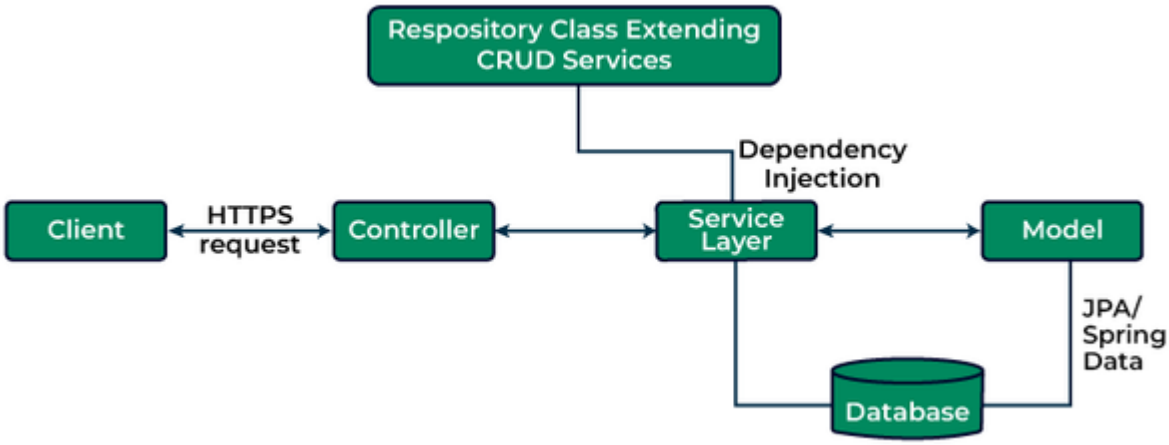
- @Repository
- JpaRepository, CrudRepository
- @Entity, @Id, @Table

4. Database Layer

The Database Layer contains the actual database where the application data is stored. It can support:

- Relational Databases (MySQL, PostgreSQL, Oracle, SQL Server).
- NoSQL Databases (MongoDB, Cassandra, DynamoDB, Firebase).
- Cloud-based databases for scalability.

Spring Boot Flow Architecture



Explanation:

- The client (frontend or API consumer) sends an HTTP request (GET, POST, PUT, DELETE) to the application.
- The request is handled by the Controller Layer, which maps the request to a specific handler method.
- The Service Layer processes business logic and communicates with the Persistence Layer to fetch or modify data.
- The Persistence Layer interacts with the Database Layer using Spring Data JPA or R2DBC, often through a **Repository Class** that extends CRUD services.
- The processed response is returned as JSON.
- Spring Boot Actuator can be used for monitoring and health checks.

Monitoring with Spring Boot Actuator

Spring Boot provides Spring Boot Actuator, which adds production-ready features to the application.

Actuator Features:

- Health checks
- Application metrics
- Environment details
- Thread and memory monitoring
- Custom management endpoints
- Actuator helps monitor application health and performance in real-time.

Spring Boot - Annotations

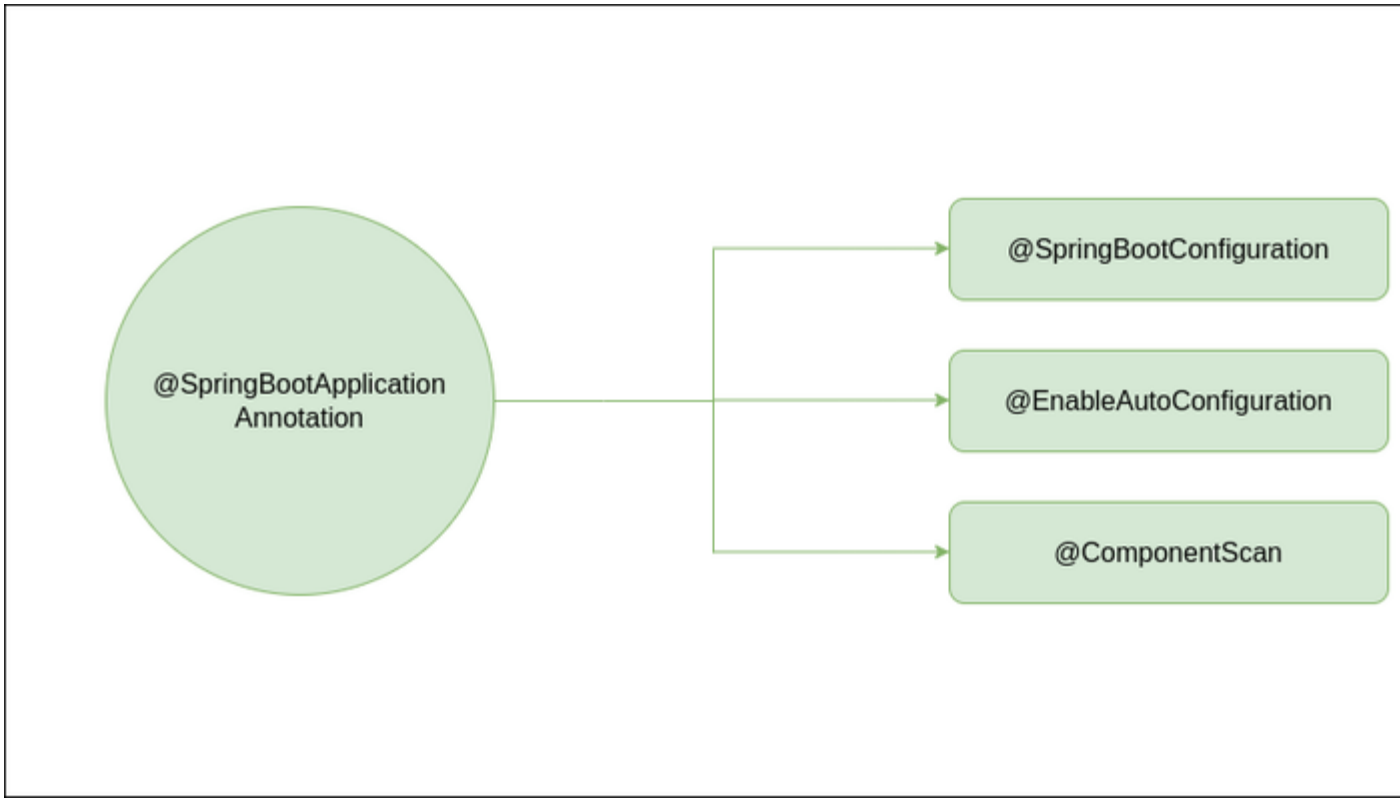
Annotations in Spring Boot are metadata that provide instructions to the Spring framework at runtime or compile time. Instead of using complex XML configurations, annotations allow developers to configure applications directly in Java code.

- Reduces XML-based configuration
- Improves code readability and maintainability
- Enables auto-configuration
- Simplifies dependency injection
- Speeds up application developmen

Core Spring Boot Annotations

1. @SpringBootApplication Annotation

- This annotation is used to mark the main class of a Spring Boot application.
- It encapsulates @SpringBootConfiguration, @EnableAutoConfiguration and @ComponentScan annotations with their default attributes.



Example:

```
@SpringBootApplication

// Class
public class DemoApplication {

  // Main driver method
  public static void main(String[] args)
  {
    SpringApplication.run(DemoApplication.class, args);
  }
}
```

2. @SpringBootConfiguration Annotation

Indicates that a class provides configuration for a Spring Boot application. It is a specialized version of @Configuration and is automatically included in @SpringBootApplication.

```
@SpringBootConfiguration
public class AppConfig {
}
```

3. @EnableAutoConfiguration Annotation

Enables Spring Boot's auto-configuration mechanism, automatically configuring beans based on classpath dependencies and application properties.

```
@EnableAutoConfiguration
public class AppConfig {
}
```

4. @ComponentScan Annotation

Tells Spring where to search for components like @Controller, @Service, @Repository, etc.

```
@ComponentScan("com.example")
public class AppConfig {
}
```

Spring Bean Annotations

1. @Component

Marks a class as a generic Spring-managed component. Spring automatically detects and registers it during component scanning.

```
@Component
public class EmailService {
}
```

2. @Service

Used in the service layer to define business logic and improve code readability.

```
@Service
public class UserService {
}
```

3. @Repository

Used in the DAO layer to interact with the database. Automatically translates database-related exceptions into Spring exceptions.

```
@Repository
public class UserRepository {
}
```

4. @Configuration

Indicates that a class contains Spring configuration and bean definitions. Acts as a replacement for XML-based configuration.

```
@Configuration

public class AppConfig {

}
```

5. @Bean

Used to define a Spring bean explicitly inside a configuration class. Gives full control over bean creation and lifecycle.

```
@Configuration
public class AppConfig {
@Bean
public RestTemplate restTemplate() {
return new RestTemplate();
}
}
```

Dependency Injection Annotations

1. @Autowired

Automatically injects required dependencies into a class. Eliminates the need for manual object creation.

```
@Autowired
private UserService userService;
```

2. @Qualifier

Specifies which bean to inject when multiple beans of the same type exist.

```
@Autowired
@Qualifier("emailService")
private NotificationService service;
```

3. @Primary

Marks a bean as the default choice among multiple candidates. Used when no @Qualifier is explicitly specified.

```
@Primary
@Component
public class SmsService implements NotificationService {
}
```

Web and REST API Annotations

1. @RestController

Used to create RESTful web services and automatically returns data in JSON or XML format.

```
@RestController
public class HelloController {
}
```

2. @RequestMapping

Maps HTTP requests to controller classes or methods. Defines the base URL path for request handling.

```
@RequestMapping("/api")
public class ApiController {
}
```

3. HTTP Method Annotations

Annotation	HTTP Method	Explanation
@GetMapping	GET	Retrieves data from the server
@PostMapping	POST	Sends data to the server
@PutMapping	PUT	Updates existing data
@DeleteMapping	DELETE	Deletes data

Example

```
@GetMapping("/users")
public List<User> getUsers() {
return userService.getAllUsers();
}
```

4. @PathVariable

Extracts values from the URI path and binds them to method parameters.

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable int id) {
return userService.getUser(id);
}
```

5. @RequestParam

Reads query parameters from the request URL. Used for optional or filtering inputs.

```
@GetMapping("/search")
public String search(@RequestParam String keyword) {
return keyword;
}
```

6. @RequestBody

Binds the HTTP request body to a Java object. Commonly used with POST and PUT requests.

```
@PostMapping("/users")
public User saveUser(@RequestBody User user) {
```

```
return userService.save(user);
}
```

Configuration and Properties Annotations

1. @Value

Injects individual property values from application.properties or application.yml.

```
@Value("${server.port}")
private String port;
```

2. @ConfigurationProperties

Binds a group of related configuration properties to a POJO in a type-safe manner.

```
@ConfigurationProperties(prefix = "app")
public class AppConfig {
private String name;
}
```

Validation Annotations

Validation annotations ensure input data correctness.

- **@NotNull** - Field value must not be null
- **@NotBlank** - String must contain at least one non-whitespace character
- **@Email** - Validates email format
- **@Size** - Restricts field length or size

Example

```
public class User {

@NotBlank

private String username;

@Email

private String email;

}
```

Exception Handling Annotations

1. @ExceptionHandler

Handles specific exceptions within a controller. Allows custom error responses for exceptions.

```
@ExceptionHandler(Exception.class)
public String handleException() {
return "Error occurred";
}
```

2. @ControllerAdvice

Provides global exception handling across all controllers. Centralizes error-handling logic.

```
@ControllerAdvice
public class GlobalExceptionHandler {
}
```

JPA and Database Annotations

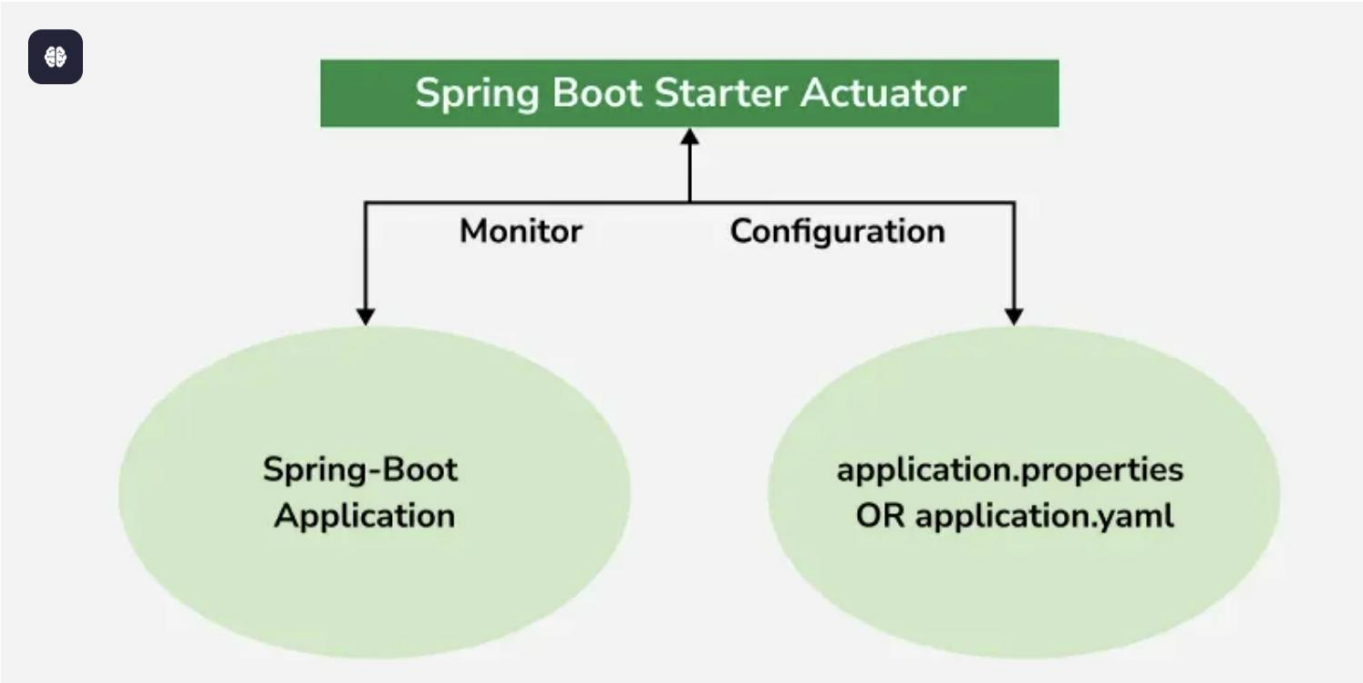
Annotation	Purpose
@Entity	Marks a class as a JPA entity
@Table	Specifies table mapping
@Id	Defines primary key
@GeneratedValue	Auto-generates primary key values
@Column	Maps class field to table column

Spring Boot Actuator

Monitoring and managing applications in production is essential to ensure stability and performance. Spring Boot Actuator provides built-in features to track application health, metrics, and internal state through easy-to-use endpoints.

- Exposes production-ready endpoints (like /actuator/health) to check application status
- Supports monitoring via HTTP endpoints and JMX integration

- Helps track metrics, performance, and system behavior for better management



Spring Boot Starter Actuator

Advantages of Actuator Application

- It increases customer satisfaction.
- It reduces downtime.
- It boosts productivity.
- It improves Cybersecurity Management.
- It increases the conversion rate.

1. Configuration for Actuator

In order to use hibernate validators, these configurations are necessary in your Spring Boot project.

1.1 Adding Actuator Dependency

To use the "Actuator", add the following dependency in your application's project settings file.

For Maven (pom.xml):

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-actuator</artifactId>

</dependency>

</dependencies>

For Gradle (build.gradle):

dependencies {

implementation 'org.springframework.boot:spring-boot-starter-actuator'

}

1.2 Configuring Actuator in application.properties

Actuator provides several configuration options to customize its behavior. Below are some common configurations:

- We can also change the default endpoint by adding the following in the application.properties file.

management.endpoints.web.base-path=/details

- Including IDs/Endpoints

By default, all IDs are set to false except for 'health'. To include an ID, use the following property in the application.properties file.

management.endpoint.<id>.enabled

Example -> management.endpoint.metrics.enabled=true

- List down all IDs that we want to include which are separated by a comma.

management.endpoints.web.exposure.include=metrics,info

- Include only metrics and info IDs and will exclude all others ('health' too).

To add/include all ID information about our application, we can do it in the application.properties file by simply adding the following

*management.endpoints.web.exposure.include=**

- Excluding IDs/Endpoints

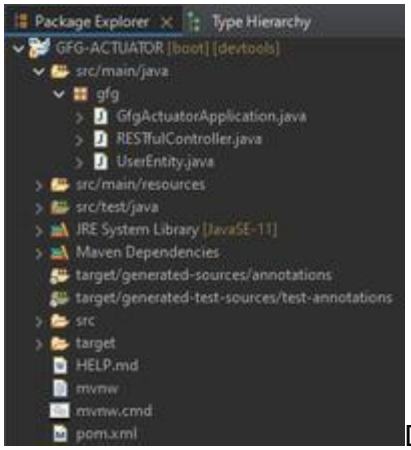
To exclude an ID or endpoint, use the following property and list out the respective IDs separated by a comma in the application.properties file.

management.endpoints.web.exposure.exclude

Example -> management.endpoints.web.exposure.exclude=info

Project Folder Structure:

The below image demonstrates the picture of how your project must look:



Directory -Structure

2. Implementing the Project

2.1 Entity Class

UserEntity.java (Entity class representing the model data) is explained below:

- This class acts as a simple java bean whose properties are returned as JSON response by the REST API's get() method.
- 'Lombok' library is used to generate GETTER/SETTER methods automatically at runtime using '@Data' annotation.
- '@RequiredArgsConstructor' annotation is used to generate a zero-argument constructor and if final or '@NonNull' fields are present, then respective arguments constructor is created.
- To add the 'Lombok' library in your application, add the following dependency in your application's project build.
- '@Component' annotation is used so that this bean automatically gets registered in Spring's application context.

Implementation:

package gfg;

import lombok.Data;

import lombok.RequiredArgsConstructor;

import org.springframework.stereotype.Component;

```
@Component
@Data
@RequiredArgsConstructor
public class UserEntity {
  String id = "1";
  String name = "Darshan.G.Pawar";
  String userName = "@drash";
  String email = "drash@geek";
  String pincode = "422-009";
}
```

2.2 Controller

RESTfulController.java (A REST API controller) for defining APIs and testing the program.

This controller's get() method uses the UserEntity bean to return JSON response. UserEntiy bean is outsourced through '*@Autowired*' annotation which was registered in Spring's application context.

```
package gfg;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/get")
public class RESTfulController {

  @Autowired
  UserEntity entity;
```

```
  @GetMapping("/data")
   public UserEntity getEntity(){
    return entity;
  }
}
```

3. Testing Actuator APIs

3.1 Controller APIs

Here, the JSON Formatter Chrome extension is used to automatically parse the JSON body. Further, it will be required to work with "Actuator".

3.2 Working with Spring Boot Actuator APIs

To access the 'Actuator' services, you will have to use the HTTP endpoint as it becomes reliable to work with.

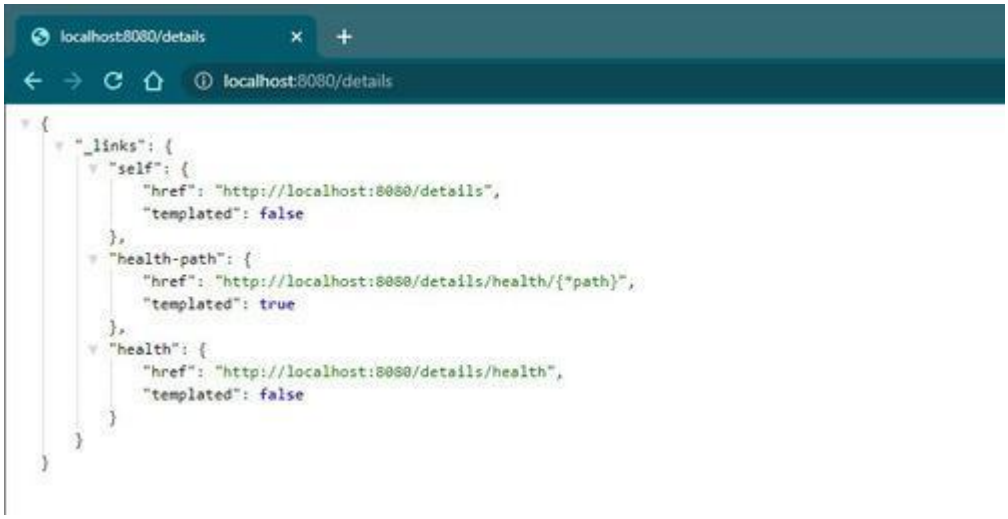
3.2.1 /actuator

It's simple just hit the default endpoint '/actuator', ensure that your Application is running.

Example:

We can also change the default endpoint by adding the following in the application.properties file.

management.endpoints.web.base-path=/details



3.2.2 /actuator/health

We can click on these above links and see the respective information. Additionally, we can activate other Actuator IDs and use them after '/actuator' to see more information. For example, 'health' ID is activated by default. Therefore we can click the link in the image or directly use 'http://localhost:8080/actuator/health'.

'UP' means the application's health is good.

Commonly Used Actuator Endpoints

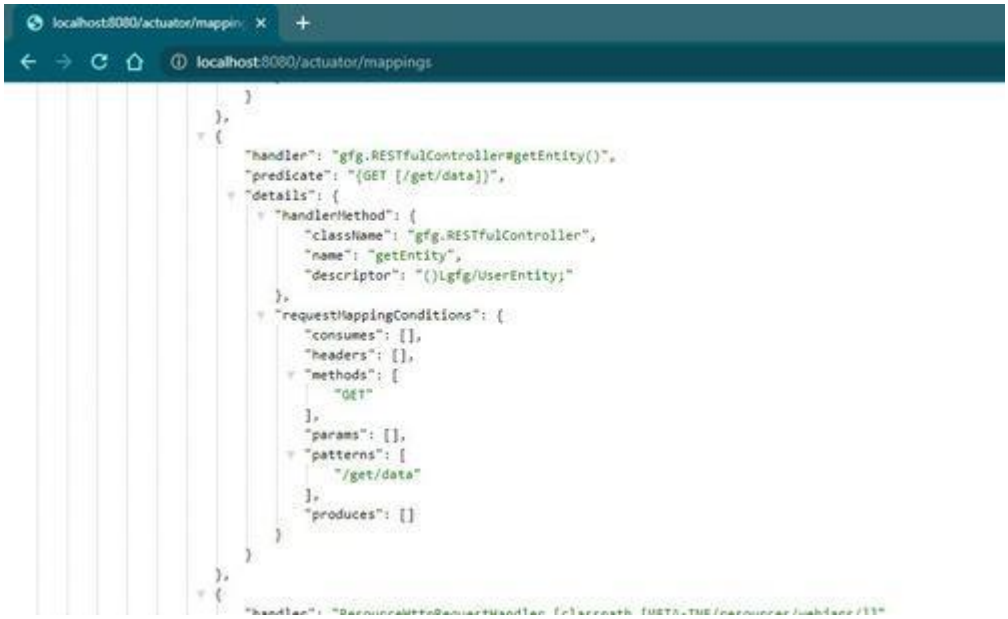
There are a total of 25 IDs out of which the commonly used are listed out below:

EndpointID	Description
Beans	Displays a complete list of all the Spring beans in your application.
Caches	Exposes available caches.
Conditions	Shows conditions evaluated on configuration and auto-configuration classes.
Health	Shows application health information.
Httptrace	Displays HTTP trace information (last 100 requests).
Loggers	Shows and modifies the configuration of loggers in the application.
Mappings	Displays a collated list of all @RequestMapping paths.

EndpointID	Description
Sessions	Retrieves and deletes user sessions (requires Spring Session).
Threaddump	Performs a thread dump.

3.2.3 /actuator/beans

3.2.4 /actuator/mappings



Including IDs/Endpoints

By default, all IDs are set to false except for 'health'. To include an ID, use the following property in the application.properties file.

management.endpoint.<id>.enabled

Example -> management.endpoint.metrics.enabled=true

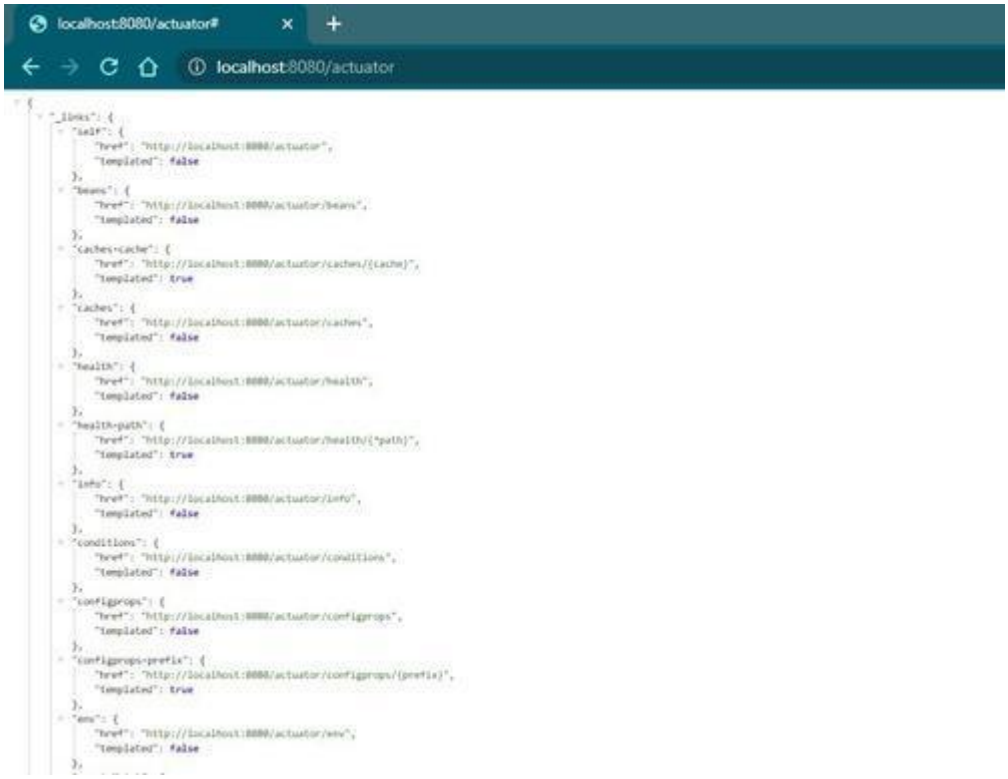
OR, we can just list down all IDs that you want to include which are separated by a comma.

management.endpoints.web.exposure.include=metrics,info

This will include only metrics and info IDs and will exclude all others ('health' too). To add/include all ID information about our application, we can do it in the application.properties file by simply adding the following

*management.endpoints.web.exposure.include=**

Output:



Excluding IDs/Endpoints

To exclude an ID or endpoint, use the following property and list out the respective IDs separated by a comma in the application.properties file.

management.endpoints.web.exposure.exclude

Example -> management.endpoints.web.exposure.exclude=info

Spring Boot - Introduction to RESTful Web Services

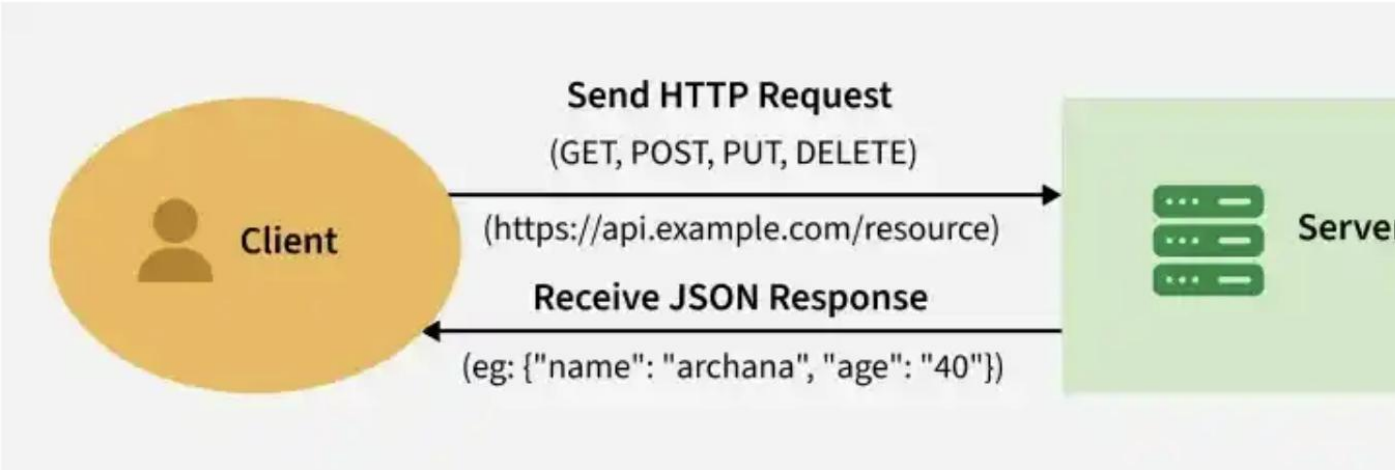
RESTful Web Services provide a standard way to build scalable and stateless web APIs using HTTP. In modern applications, REST APIs allow different systems such as web apps, mobile apps, and microservices to communicate efficiently.

- Spring Boot simplifies the creation of REST APIs using annotations and minimal configuration.
- REST services exchange data in formats such as JSON and XML, with JSON being the most widely used format.

What is REST?

REST (Representational State Transfer) is an architectural style used for designing network-based applications. It allows different systems to communicate over HTTP using standard methods such as GET, POST, PUT, and DELETE.

- Introduced by Roy Thomas Fielding to simplify web communication using HTTP protocols.
- Widely used for building scalable and stateless web APIs in frameworks like Spring Boot.



REST

Key Concepts of RESTful Web Services

1. Resource

A resource is any object, data, or service that can be accessed by a client using a URI (Uniform Resource Identifier).

Example:

/users
/products
/orders

2. Stateless Communication

REST APIs are stateless, meaning every request from the client must contain all necessary information. The server does not store client session data between requests.

3. Representations

Resources can be represented in multiple formats when returned to the client. Common formats include:

- JSON
- XML
- HTML
- PDF

4. HTTP Verbs

REST APIs use standard HTTP methods to perform CRUD operations on resources.

HTTP Methods in REST APIs

The main methods of HTTP we build web services for are.

1. GET – Read Resource

- Retrieves data without a request body.
- Can fetch a specific resource using an ID or a collection without parameters.

Spring Boot Example:

```
@GetMapping("/user/{userId}")
public ResponseEntity<UserEntity> getUser(@PathVariable int userId) {
  UserEntity user = userService.getUser(userId);
  return ResponseEntity.ok(user);
}
```

2. POST – Create Resource

- Creates a new resource using a request body.

Spring Boot Example:

```
@PostMapping("/user")
public ResponseEntity<String> addUser(@RequestBody UserEntity user) {
  userService.saveOrUpdate(user);
  return ResponseEntity.status(HttpStatus.CREATED).body("User created
successfully");
}
```

3. PUT – Update Resource

- Updates an existing resource identified by ID.

Spring Boot Example:

```
@PutMapping("/user/{userId}")
public ResponseEntity<String> updateUser(@PathVariable int userId, @RequestBody UserEntity user) {
  userService.saveOrUpdate(user);
  return ResponseEntity.ok("User updated successfully");
}
```

4. DELETE – Remove Resource

- Deletes a single or multiple resources based on parameters.

Spring Boot Example:

```
@DeleteMapping("/user/{userId}")
public ResponseEntity<String> deleteUser(@PathVariable int userId) {
  userService.deleteUser(userId);
  return ResponseEntity.ok("User deleted successfully");
}
```

HTTP Method	Operation	Description
GET	Read	Retrieves existing data
POST	Create	Creates a new resource
PUT	Update	Updates an existing resource
DELETE	Delete	Removes a resource

HTTP Status Codes

REST APIs rely on these codes to communicate the result of client requests.

Status Code	Meaning
200	Request successful
201	Resource created
401	Unauthorized access
404	Resource not found
500	Internal server error

Principles of RESTful Web Services

- **Resource Identification via URI:** Every resource has a unique URI.
- **Uniform Interface:** CRUD operations use standard HTTP methods: GET, POST, PUT, DELETE.
- **Self-Descriptive Messages:** The request and response contain all necessary information.
- **Stateless Interactions:** Each request is independent; no session data is stored on the server.
- **Cacheable:** Responses can be cached when appropriate to improve performance.

Security Best Practices for REST APIs

- **Authentication and Authorization:** Use JWT or OAuth 2.0.
- **Input Validation:** Sanitize requests to prevent SQL injection and XSS attacks.
- **HTTPS Enforcement:** Ensure all communications are encrypted.
- **Rate Limiting:** Protect against abuse by limiting request rates.

Advantages of RESTful Web Services

- **Simple and Lightweight:** Easier to develop and consume compared to SOAP.
- **Client-Server Decoupling:** Enables independent development of client and server.

- **Scalable:** Stateless communication supports horizontal scaling.
- **Layered System Architecture:** Applications can be divided into layers, enhancing modularity and maintainability.
- **Cacheable:** Responses can be cached to improve performance and reduce bandwidth.

Uses of REST with Spring Boot

Spring Boot makes building RESTful APIs fast and efficient by:

- Simplifying configuration and setup.
- Providing out-of-the-box support for JSON and XML serialization.
- Allowing integration with databases, messaging systems, and external APIs.
- Supporting advanced features like validation, exception handling, and security.